

A

Project report on

API Testing & Management Platform

Submitted in partial fulfillment of the requirement for the award of the degrees of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY



Session: 2025-26

Submitted by:

Ayachi Sharma (Enrl. No. - 22E1MBITM40P009)

Akshita Soni (Enrl. No. - 22E1MBITM40P007)

Dilip Singh Bhati (Enrl No.- 22E1MBITM40P014)

Dakshansh Dadhich (Enrl. No. - 22E1MBITM40P012)

Under the guidance of:

Mrs. Mridula Maheshwari

Submitted to:

Mr. Anurag Jagetia

Head of Department

Information Technology

MLV Textile & Engineering College

Pratap Nagar, Pur Road,
Bhilwara 311001, Rajasthan

Declaration by Student(s)

I hereby declare that the work presented in this project titled "API Testing & Management Platform" submitted towards completion of project in 8th Semester of B. Tech (IT) at the M.L.V. Textile and Engineering College, Bhilwara is an authentic record of my original work pursued under the guidance of Mrs.Mridula Maheshwari **M.L.V. Textile & Engineering College, Bhilwara.**

I have not submitted the matter embodied in this project for the award of any other degree to any other institute/college.

I have not submitted the matter embodied in this project for the award of any other degree to any other institute/college.

Signature and name of the student(s) with date

Ayachi Sharma

Akshita Soni

Dilip Singh Bhati

Dakshansh Dadhich



M. L. V. Textile & Engineering College, Bhilwara
(A constituent College of Rajasthan Technical University, Kota)



Information Technology

BONAFIDE CERTIFICATE

This is to certify that the project work entitled “API Testing & Management Platform”

Ayachi Sharma (22EMBIT009)

Akshita Soni (22EMBIT007)

Dilip Singh Bhati (22EMBIT014)

Dakshansh Dadhich (22EMBIT012)

under my supervision and guidance during the academic year 2025-26. To the best of my knowledge, the project work embodied in this report is an original piece of work.

Project Guide

Mrs. Mridula Maheshwari

Date:

Submitted for viva-voce examination

External Examiner 1

(Name, Designation of
External Examiner 1)

Acknowledgement

We delightfully thank **Mr. Arvind Vashisht**, Principal, M. L. V. Textile & Engineering College, Bhilwara, for permitting us to carry out this project work.

We extend our sincere thanks to **Mr. Anurag Jagetia**, Head of Department (HOD) and **Mrs. Neha Mangal**, Project Incharge for his kind cooperation and guidance throughout the project.

We pay our special thanks to **Mrs. Mridula Maheshwari**, Project Guide, for her constant encouragement, valuable suggestions, and support that we have received during the entire course of the project.

We are also thankful to all the faculty members for their generous support and for being a constant source of motivation and inspiration.

Finally, yet importantly, we would like to express our heartfelt thanks to our beloved parents for their blessings, and to our friends/classmates for their help and best wishes for the successful completion of this project.

Ayachi Sharma

Akshita Soni

Dilip Singh Bhati

Dakshansh Dadich

B. Tech. IV Year

Abstract

It is a comprehensive, web-based API testing and management platform designed for developers, QA engineers, and API developers to test, debug, and manage REST APIs with ease. The platform addresses the growing need for efficient API testing tools in modern software development workflows. Built with modern technologies including React, TypeScript, Vite for the frontend and Node.js with Express.js for the backend, this application provides an intuitive interface for making HTTP requests, analysing responses, and maintaining API collections within a single, cohesive dashboard.

The platform combines the functionality of established tools like Postman with integrated AI powered insights, allowing users to test APIs, view execution history, and generate automatic test results. It implements a full-stack solution with user authentication using JWT tokens, request management with MongoDB database storage, and real-time response visualization. The system supports multiple HTTP methods including GET, POST, PUT, PATCH, and DELETE with comprehensive parameter and header management capabilities.

Key innovations include automatic test generation powered by AI, performance metrics tracking, and an interactive chat interface for API testing assistance. The backend architecture follows RESTful principles with proper error handling, input validation using Zod, and secure authentication mechanisms. The experimental results demonstrate efficient API request handling with response times under 200ms for typical requests and successful integration of AI-powered test case generation.

In addition, focuses on user experience and accessibility by offering a clean, responsive, and userfriendly interface that simplifies API testing for both beginners and experienced developers. The integration of AI-driven recommendations helps users identify errors, suggest improvements, and generate meaningful test cases automatically, reducing manual effort and improving testing accuracy. Future enhancements of the platform may include support for GraphQL APIs, automated CI/CD pipeline integration, and enhanced security testing features, making a robust and scalable solution for modern API development and testing needs.

Keywords: API Testing, REST API, React, Node.js, Express.js, MongoDB, JWT Authentication, AI-powered Testing, Web Application, Full-stack Development

Table of Contents

Cover Page	I
Declaration by student(s).....	II
Bonafide	III
Acknowledgement	IV
Abstract.....	V
List of Tables.....	VII
Nomenclature and abbreviations.....	IX

1: Introduction

- 1.1 Motivation
- 1.2 Aim of the Project
- 1.3 Scope of the Work
- 1.4 Problem Statement
- 1.5 Objectives

2: Literature Review

- 2.1 Evolution of API Testing
- 2.2 Existing API Testing Tools
- 2.3 AI in Software Testing
- 2.4 Web Application Architecture
- 2.5 Database Technologies
- 2.6 Authentication and Security
- 2.7 Summary of Literature Review

3: Requirements for Project Completion

- 3.1 Project Functional Requirements
- 3.2 Project Non-functional Requirements
- 3.3 User Interface Requirements
- 3.4 Hardware Interface Requirements
- 3.5 Software Interface Requirements

4: Proposed Approach

- 4.1 Planning
 - 4.1.2 Resource Graph for Time-Effort Estimation
- 4.2 Design
 - 4.2.1 System Architecture
 - 4.2.2 Data Flow Diagram
 - 4.2.3 Database Schema Design
 - 4.2.4 Use Case Diagrams
 - 4.2.5 Class Diagram
 - 4.2.6 Activity Diagram
 - 4.2.7 Sequence Diagram
- 4.3 Algorithms

5: Experimental Setup

- 5.1 Development Environment Setup
- 5.2 Frontend Implementation
- 5.3 Backend Implementation
- 5.4 Database Configuration
- 5.5 Performance Analysis

6: Results and Discussion

- 6.1 User Interface Results

6.2 Functional Test Results

6.3 Performance Test Results

6.4 Security Analysis

6.5 Discussion

7: Conclusion and Future Scope

7.1 Conclusion

7.2 Future Scope

7.3 Applications

8: References

List of Tables & Nomenclatures

Table No.	Description
Table 3.1	Functional Requirements Specification
Table 3.2	Non-functional Requirements
Table 3.3	Hardware Requirements
Table 3.4	Software Requirements
Table 5.1	Frontend Dependencies
Table 5.2	Backend Dependencies

Nomenclature and Abbreviations

• Abbreviation	Full Form
• API	Application Programming Interface
• REST	Representational State Transfer
• HTTP	Hypertext Transfer Protocol
• JSON	JavaScript Object Notation
• JWT	JSON Web Token
• CRUD	Create, Read, Update, Delete
• UI	User Interface
• UX	User Experience
• DOM	Document Object Model
• SPA	Single Page Application
• MVC	Model-View-Controller
• ORM	Object-Relational Mapping
• ODM	Object Document Mapper
• CSS	Cascading Style Sheets
• HTML	Hypertext Markup Language
• NPM	Node Package Manager
• IDE	Integrated Development Environment
• QA	Quality Assurance
• AI	Artificial Intelligence
• URL	Uniform Resource Locator

Chapter 1: Introduction

1.1 Motivation

In the contemporary software development landscape, APIs have become the backbone of modern applications, enabling seamless communication between different software components, services, and platforms. With the proliferation of microservices architecture and cloud-based applications, the importance of robust API testing has grown exponentially. Developers and QA engineers spend significant time testing APIs manually, which is both time-consuming and error-prone.

Existing API testing tools, while functional, often present challenges such as steep learning curves, high licensing costs, limited collaboration features, or lack of intelligent testing capabilities. The motivation behind stems from the need to create an accessible, feature-rich, and intelligent API testing platform that addresses these limitations.

The integration of AI-powered testing capabilities represents a significant advancement over traditional API testing tools. By automatically generating test cases and providing intelligent insights, reduces the cognitive load on developers and helps identify potential issues that might be overlooked during manual testing.

Furthermore, the web-based nature of the platform eliminates installation requirements and enables cross-platform accessibility, making it convenient for distributed development teams to collaborate effectively.

1.2 Aim of this Project

The primary aim of this project is to design and develop a comprehensive, web-based API testing and management platform that provides:

1. **Intuitive API Testing Interface:** Create a user-friendly interface for making HTTP requests with support for multiple HTTP methods, custom headers, query parameters, and request bodies.
2. **Intelligent Test Generation:** Implement AI-powered automatic test case generation that analyzes API responses and provides meaningful test results with explanations.
3. **Request History Management:** Develop a robust system for tracking and managing API request history, enabling users to revisit and reuse previous requests.
4. **Collection Organization:** Build functionality for organizing API endpoints into logical collections for better management and accessibility.
5. **Secure Authentication System:** Implement a secure user authentication system using JWT tokens to protect user data and enable personalized experiences.
6. **Real-time Response Analysis:** Provide comprehensive response analysis including status codes, response times, payload sizes, and formatted response bodies.
7. **Schema Generation:** Develop automatic API schema generation capabilities for documentation purposes.

1.3 Scope of the Work

The scope of this project encompasses the complete development lifecycle of a full-stack web application, including:

Frontend Development:

- Development of a responsive single-page application using React and TypeScript
- Implementation of state management using Zustand
- Integration of UI components using shadcn/ui library
- Form handling with React Hook Form and Zod validation
- Data fetching and caching using TanStack React Query

Backend Development:

- Development of RESTful API services using Node.js and Express.js
- Implementation of MongoDB database integration using Mongoose ODM
- JWT-based authentication and authorization
- Input validation and error handling
- AI service integration for test generation

Integration and Testing:

- API integration between frontend and backend
- Unit testing and integration testing
- Performance optimization
- Security implementation

1.4 Problem Statement

Developers and QA engineers face several challenges when testing APIs:

1. Complexity of Existing Tools: Many existing API testing tools have complex interfaces that require significant time to learn and master.
2. Manual Test Case Creation: Creating test cases manually is time-consuming and requires expertise in understanding expected API behaviors.
3. Limited Collaboration: Desktop-based tools limit collaboration among distributed team members.

4. High Costs: Enterprise API testing tools often come with substantial licensing costs that may not be feasible for small teams or individual developers.
5. Lack of Intelligent Insights: Traditional tools provide raw responses without intelligent analysis or suggestions for improvement.
6. Poor Organization: Managing multiple API endpoints across different projects without proper organization leads to inefficiency.

The problem statement can be summarized as: "How to develop an accessible, intelligent, and comprehensive API testing platform that simplifies the API testing workflow while providing automated insights and efficient request management?"

1.5 Objectives

The specific objectives of this project are:

1. To design and implement a responsive web-based user interface for API testing that supports multiple HTTP methods (GET, POST, PUT, PATCH, DELETE).
 2. To develop a secure backend API service with user authentication and authorization using JWT tokens.
 3. To implement a MongoDB database schema for storing user data, API requests, collections, and test results.
 4. To integrate AI-powered automatic test case generation that analyzes API responses and provides meaningful insights.
 5. To create a comprehensive request history system with pagination and filtering capabilities.
 6. To develop a collection management system for organizing and categorizing API endpoints.
 7. To implement real-time response visualization with syntax highlighting and performance metrics.
 8. To ensure the application meets security best practices including input validation, secure authentication, and data protection.
-

Chapter 2: Literature Review

The field of API testing has evolved significantly over the past decade, with numerous tools and methodologies emerging to address the growing complexity of modern software systems. This chapter reviews existing literature and tools relevant to API testing and management.

2.1 Evolution of API Testing

The concept of API testing has its roots in the early days of web services when SOAP-based services dominated enterprise integrations. Canfora and Di Penta [1] provided a comprehensive overview of service-oriented architecture testing, highlighting the unique challenges posed by loosely coupled systems. Their work emphasized the importance of contract testing and the need for automated validation of service interfaces.

With the rise of REST APIs, testing methodologies evolved to accommodate the stateless nature and resource-oriented design of RESTful services. Fielding's dissertation [2] on REST architectural style laid the foundation for understanding how APIs should be designed and, consequently, how they should be tested.

2.2 Existing API Testing Tools

Postman: Postman [3] is one of the most widely used API testing tools, offering a comprehensive suite of features including request building, automated testing with JavaScript, and team collaboration. While powerful, Postman's extensive feature set can be overwhelming for beginners, and advanced features require paid subscriptions.

Insomnia: Insomnia [4] provides a cleaner, more focused interface for API testing. It supports GraphQL natively and offers plugins for extensibility. However, it lacks some of the collaborative features present in Postman.

Swagger/OpenAPI: The OpenAPI Specification [5] and associated Swagger tools provide a standardized approach to API documentation and testing. Swagger UI enables interactive API documentation, while Swagger Codegen facilitates client SDK generation.

REST Assured: REST Assured [6] is a Java-based library that provides a domain-specific language for testing REST services. It is particularly popular in enterprise environments where Java is the primary development language.

2.3 AI in Software Testing

The application of artificial intelligence in software testing has gained significant attention. Bertolino surveyed software testing research over the past decades and identified AI-assisted testing as an emerging trend. Machine learning techniques have been applied to test case generation, test prioritization, and defect prediction.

Recent advances in large language models (LLMs) have opened new possibilities for intelligent test generation. Brown et al. [8] demonstrated the capabilities of GPT-3 in understanding and generating code, which has implications for automated test case creation.

2.4 Web Application Architecture

Modern web applications typically follow a client-server architecture with clear separation between frontend and backend components. The React library [9] has emerged as a popular choice for building user interfaces due to its component-based architecture and virtual DOM implementation.

On the backend, Node.js [10] has gained popularity for building scalable network applications. The event-driven, non-blocking I/O model makes it particularly suitable for I/O-intensive applications like API services.

2.5 Database Technologies

MongoDB [11] represents a NoSQL approach to data storage, offering flexibility in schema design and horizontal scalability. For applications dealing with varied data structures like API requests and responses, document-based databases provide advantages over traditional relational databases.

2.6 Authentication and Security

JSON Web Tokens (JWT) [12] have become a standard for stateless authentication in web applications. The self-contained nature of JWTs allows for scalable authentication without serverside session storage.

2.7 Summary of Literature Review

The literature review highlights several widely used API testing tools and technologies, each with its own strengths and limitations. Postman is known for its comprehensive feature set and strong team collaboration capabilities, making it a popular choice among developers; however, it has a relatively complex interface and many advanced features are available only in paid versions. The literature review reveals a gap in the market for an accessible, web-based API testing tool that combines ease of use with intelligent testing capabilities. aims to address this gap by providing a modern, AI-powered platform that simplifies the API testing workflow.

Chapter 3: Requirements for Project Completion

This chapter details the comprehensive requirements for the platform, categorized into functional, non-functional, user interface, hardware, and software requirements.

3.1 Project Functional Requirements

Functional requirements define the specific behaviors and functions of the system. The following table outlines the key functional requirements:

Table 3.1: Functional Requirements Specification

Req. ID	Requirement Description	Priority
FR-01	The system shall allow users to register with email and password	High
FR-02	The system shall authenticate users using JWT tokens	High
FR-03	The system shall support HTTP methods: GET, POST, PUT, PATCH, DELETE	High
FR-04	The system shall allow users to add custom headers to requests	High
FR-05	The system shall support query parameters with enable/disable toggles	High
FR-06	The system shall allow JSON request body for POST, PUT, PATCH methods	High
FR-07	The system shall display response status code, headers, and body	High
FR-08	The system shall track and display response time	Medium
FR-09	The system shall maintain request history for each user	High
FR-10	The system shall allow users to create and manage collections	Medium
FR-11	The system shall generate automatic test cases using AI	High
FR-12	The system shall provide syntax highlighting for JSON responses	Medium
FR-13	The system shall support Bearer token authentication for requests	High

FR-14	The system shall allow users to save requests to collections	Medium
FR-15	The system shall provide pagination for request history	Low
FR-16	The system shall auto-generate API schemas	Medium
FR-17	The system shall provide an AI chat interface for assistance	Medium

3.2 Project Non-functional Requirements

Non-functional requirements specify the quality attributes and constraints of the system.

Table 3.2: Non-functional Requirements

Req. ID	Category	Requirement Description
NFR-01	Performance	API response time shall be less than 200ms for internal operations
NFR-02	Performance	The system shall handle at least 100 concurrent users
NFR-03	Scalability	The architecture shall support horizontal scaling
NFR-04	Security	All passwords shall be hashed using bcrypt
NFR-05	Security	JWT tokens shall expire after 24 hours
NFR-06	Security	All API endpoints shall validate input data
NFR-07	Reliability	The system shall have 99.9% uptime
NFR-08	Usability	The interface shall be intuitive and require minimal training
NFR-09	Compatibility	The application shall work on Chrome, Firefox, Safari, and Edge
NFR-10	Maintainability	Code shall follow established coding standards and be well documented

NFR-11	Accessibility	The interface shall follow WCAG 2.1 guidelines
--------	---------------	--

3.3 User Interface Requirements

The user interface requirements define the visual and interaction design specifications:

1. Dashboard Layout: The main dashboard shall display the API request interface prominently with easy access to history and collections.
2. Request Builder: The request builder shall include:
 - HTTP method selector (dropdown)
 - URL input field
 - Tabs for Parameters, Headers, Body, and Authentication
 - Send button with visual feedback
3. Response Display: The response section shall show:
 - Status code with color coding (green for 2xx, red for 4xx/5xx)
 - Response time in milliseconds
 - Response size in bytes/KB
 - Formatted JSON response with syntax highlighting
4. History Panel: A collapsible side panel showing recent requests with:
 - Method indicator
 - URL truncation for long URLs
 - Timestamp
 - Quick replay functionality
5. Test Results: Test results shall be displayed as cards with:
 - Pass/Fail indicator
 - Test description
 - Explanation of result
 - Suggestions for improvement
6. Responsive Design: The interface shall adapt to different screen sizes:

- Desktop: Full layout with side panels
- Tablet: Collapsible panels
- Mobile: Stacked layout with navigation

3.4 Hardware Interface Requirements

Table 3.3: Hardware Requirements

Development Environment:

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or higher
Storage	20 GB free space	50 GB SSD
Display	1366 x 768 resolution	1920 x 1080 or higher
Network	Broadband internet connection	High-speed internet

3.5 Software Interface Requirements

Table 3.4: Software Requirements

Development Tools:

Software	Version	Purpose
npm	9.x or higher	Package management

Visual Studio Code	Latest	IDE
Git	2.x	Version control
MongoDB	6.x	Database
Node.js	18.x or higher	Runtime environment
Postman	Latest	API testing during development

Frontend Dependencies:

Package	Version	Purpose
React	18.x	UI library
TypeScript	5.x	Type checking
Vite	5.x	Build tool
Tailwind CSS	3.x	Styling
Zustand	4.x	State management
React Query	5.x	Server state management
React Router	6.x	Routing
Axios	1.x	HTTP client
Zod	3.x	Validation

React Hook Form	7.x	Form handling
-----------------	-----	---------------

Backend Dependencies:

Package	Version	Purpose
Express.js	4.x	Web framework
Mongoose	8.x	MongoDB ODM
jsonwebtoken	9.x	JWT handling
bcryptjs	2.x	Password hashing
cors	2.x	CORS middleware
dotenv	16.x	Environment variables
express-validator	7.x	Input validation
helmet	7.x	Security headers

Chapter 4: Proposed Approach

This chapter presents the detailed design and planning approach for the platform, including project scheduling, system architecture, and various design diagrams.

4.1 Planning

The project was planned over a period of 16 weeks, divided into the following phases:

Phase 1: Requirements and Design (Weeks 1-3)

- Week 1: Requirements gathering and analysis
- Week 2: System design and architecture planning
- Week 3: Database schema design and UI mockups

Phase 2: Backend Development (Weeks 4-7)

- Week 4: Project setup and authentication module
- Week 5: User management and API request handling
- Week 6: Collection management and history tracking
- Week 7: AI integration and testing

Phase 3: Frontend Development (Weeks 8-12)

- Week 8: Project setup and routing configuration
- Week 9: Authentication UI and state management
- Week 10: Request builder and response display
- Week 11: History and collection management UI
- Week 12: AI panel and schema generator

Phase 4: Integration and Testing (Weeks 13-15)

- Week 13: Frontend-backend integration
- Week 14: Testing and bug fixing
- Week 15: Performance optimization and security audit

Phase 5: Documentation and Deployment (Week 16)

- Documentation completion
- Deployment preparation
- Final review and submission

4.1.2 Resource Graph for Time-Effort Estimation

The project required the following resource allocation:

Resource	Hours/Week	Total Hours
Project Planning	10	40
Backend Development	25	100
Frontend Development	30	150
Database Design	8	24
Testing	15	45
Documentation	10	40
Total	-	399

4.2 Design

4.2.1 System Architecture

The platform follows a three-tier architecture consisting of:

Presentation Layer (Frontend)

- React-based single-page application
- Component-based architecture
- State management with Zustand
- API communication via Axios

Application Layer (Backend)

- Express.js REST API server

Route handlers for different modules

- Middleware for authentication and validation
- Business logic services

Data Layer

- MongoDB database
- Mongoose ODM for data modeling
- Collections for users, requests, history, and collections

The architecture diagram illustrates the flow of data from the client through the API gateway to the database, with the AI service integrated for test generation.

4.2.2 Data Flow Diagram

Level 0 DFD (Context Diagram)

The context diagram shows the system as a single process interacting with external entities:

- User: Interacts with the system by sending API requests and receiving responses
- External API: The target API being tested
- AI Service: Provides test case generation capabilities

Level 1 DFD

The Level 1 DFD expands the system into major processes:

1. User Authentication Process: Handles registration, login, and token validation
2. Request Management Process: Processes API requests and manages parameters/headers
3. History Management Process: Stores and retrieves request history
4. Collection Management Process: Organizes requests into collections
5. Test Generation Process: Interfaces with AI service for automatic testing
6. Response Analysis Process: Formats and analyzes API responses

4.2.3 Database Schema Design

The MongoDB database is designed using a document-oriented approach and consists of the following collections:

1. Users Collection

The *Users* collection stores information related to registered users of the system. Schema:

requests: Array of ObjectIds (Reference to Requests collection)

- createdAt: Date
- updatedAt: Date

4.2.4 Use Case Diagrams

User Authentication Use Case Diagram

Actors: User, System

Use Cases:

- UC-01: Register Account
- UC-02: Login
- UC-03: Logout
- UC-04: Reset Password
- UC-05: View Profile
- UC-06: Update Profile

Relationships:

- Register includes Email Validation
- Login includes Token Generation
- All authenticated use cases extend from Verify Token

API Request Management Use Case Diagram

Actors: User, External API, AI Service

Use Cases:

- UC-07: Create New Request
- UC-08: Configure Headers
- UC-09: Add Query Parameters
- UC-10: Set Request Body
- UC-11: Configure Authentication
- UC-12: Send Request
- UC-13: View Response

UC-14: Generate Test Cases

- UC-15: Save to Collection
- UC-16: View History

Relationships:

- Send Request includes External API Call
- View Response extends Generate Test Cases
- Generate Test Cases includes AI Service Integration

4.2.5 Class Diagram

The class diagram represents the object-oriented structure of the application:

Frontend Classes

Api Request:

- method: Http Method
- url: string
- headers: Header[]
- params: Param[]
- body: string
- auth Type: Auth Type
- auth Token: string
+ send(): Promise<ApiResponse> +
validate(): boolean

Api Response:

- status: number
- statusText: string
- headers: Map<string, string>
- body: any
- time: number
- size: number
+ format(): string
+ getTestResults(): TestResult[]

User:

- id: string

- email: string
- name: string
- token: string
- + login(credentials): Promise<void>
- + register(data): Promise<void>
- + logout(): void

Collection:

- id: string
- name: string
- description: string
- requests: ApiRequest[]
- + addRequest(request): void
- + removeRequest(requestId): void
- + save(): Promise<void>

Backend Classes / Models

UserModel:

- + create(userData): Promise<User>
- + findByEmail(email): Promise<User>
- + findById(id): Promise<User>
- + updatePassword(id, password): Promise<void>
- + comparePassword(password): Promise<boolean>

RequestModel:

- + create(requestData): Promise<Request>
- + findByUserId(userId): Promise<Request[]>
- + findById(id): Promise<Request>
- + update(id, data): Promise<Request>
- + delete(id): Promise<void>

HistoryModel:

- + create(historyData): Promise<History>
- + findByUserId(userId, options): Promise<History[]>
- + findById(id): Promise<History>
- + deleteOld(userId, days): Promise<void>

4.2.6 Activity Diagram

Activity Diagram for API Testing Flow

1. Start
2. User opens application
3. Decision: Is user authenticated?
 - No → Navigate to login page → Enter credentials → Validate credentials
 - Invalid → Show error → Return to login
 - Valid → Generate JWT → Store token → Proceed
 - Yes → Proceed
4. Display dashboard
5. User enters API URL
6. Decision: Configure request?
 - Yes → Select HTTP method → Add headers (optional) → Add parameters (optional) → Set body (if applicable) → Configure auth (optional)
 - No → Proceed with defaults
7. Click Send button
8. Validate request
9. Decision: Valid request?
 - No → Show validation errors → Return to request configuration
 - Yes → Proceed
10. Send request to external API
11. Receive response
12. Fork: Parallel processing
 - Parse and format response
 - Calculate metrics (time, size)
 - Send to AI for test generation
13. Join: Combine results
14. Display response with test results
15. Decision: Save to history?

- Yes → Store in database
16. Decision: Save to collection?
- Yes → Select/create collection → Add request
17. End

4.2.7 Sequence Diagram

Sequence Diagram for Request Execution

Participants: User, Frontend, Backend API, MongoDB, External API, AI Service

1. User → Frontend: Enter request details
2. User → Frontend: Click Send
3. Frontend → Frontend: Validate input
4. Frontend → Backend API: POST /api/requests/execute
5. Backend API → Backend API: Validate JWT
6. Backend API → External API: Forward HTTP request
7. External API → Backend API: Return response
8. Backend API → AI Service: Analyze response
9. AI Service → Backend API: Return test results
10. Backend API → MongoDB: Save to history
11. MongoDB → Backend API: Confirm save
12. Backend API → Frontend: Return response + test results
13. Frontend → Frontend: Format and render response
14. Frontend → User: Display results

4.3 Algorithms

Algorithm 1: JWT Authentication

Input: Email, Password

Output: JWT Token or Error

1. Begin

2. Retrieve user using email
3. If user does not exist:
 1. Return error 'User not found'
4. End If
5. Validate password with stored hashed password
6. If password is invalid:
 2. Return error 'Invalid password'
7. End If
8. Create payload with userId and email
9. Generate JWT token with expiration time
10. Return generated token
11. End

Algorithm 2: API Request Execution

Input: RequestConfig (method, URL, headers, params, body, auth)

Output: Response Object with Test Results

1. Begin
2. Record start time
3. If authentication type is Bearer:
 1. Add Authorization header with token
4. End If
5. Construct final URL using query parameters
6. Try:
 - a. Send HTTP request using method, URL, headers, and body
 - b. Record end time
 - c. Create response object with status, headers, body, time, and size
 - d. Generate test results using AI service
 - e. Attach test results to response object
 - f. Save request and response to history
 - g. Return response object
7. Catch error:
 2. Return error message and status
8. End Try
9. End

Algorithm 3: AI Test Generation

Input: API Response (status, headers, body, time)

Output: List of Test Results

1. Begin
 2. Initialize empty testResults list
 3. Test 1: Status Code Validation
 4. Check if status is between 200 and 299
 5. If false, suggest checking API endpoint
 6. Test 2: Response Time Check
 7. Check if response time is less than 1000 ms
 8. If false, suggest optimization
 9. Test 3: Response Body Validation
 10. Check if body is not empty and valid JSON
 11. Test 4: Content-Type Validation
 12. Check if header contains 'application/json'
 13. Call AI service for additional test cases
 14. Append all test results
 15. Return testResults
 16. End
-

Chapter 5: Experimental Setup

This chapter describes the development environment setup, implementation details for both frontend and backend, database configuration, and performance analysis.

5.1 Development Environment Setup

The development environment was configured with the following tools and configurations:

Operating System: Windows 11 / macOS Ventura / Ubuntu 22.04

Node.js Installation:

- Version: 18.17.0 (LTS)
- Package Manager: npm 9.6.7

MongoDB Setup:

- Local development: MongoDB Community Server 6.0
- Production: MongoDB Atlas (Cloud)

IDE Configuration:

- Visual Studio Code with extensions:
- ESLint
- Prettier
- TypeScript Vue Plugin
- Tailwind CSS IntelliSense
- MongoDB for VS Code

Version Control:

- Git 2.41.0
- GitHub for repository hosting

5.2 Frontend Implementation

Project Structure (Frontend)

frontend/

- |— src/
 - | |— components/
 - | | |— ui/ (shadcn/ui components)
 - | | |— RequestBuilder/ (API request components)
 - | | |— ResponseViewer/ (Response display components)
 - | | |— History/ (History panel components)
 - | | |— Collections/ (Collection management)
 - | | |— Auth/ (Authentication components)
 - | | |— AIPanel/ (AI chat interface)
 - | |— hooks/ (Custom React hooks)
 - | |— lib/ (Utility functions)
 - | |— services/ (API service functions)
 - | |— store/ (Zustand store definitions)
 - | |— types/ (TypeScript definitions)
 - | |— pages/ (Page components)
 - | |— App.tsx (Root component)
 - | |— main.tsx (Entry point)
- |— public/ (Static assets)
- |— package.json
- |— tailwind.config.js
- |— tsconfig.json
- |— vite.config.ts

Table 5.1: Frontend Dependencies

Category	Package	Version	Purpose
Core	React	18.x	UI library
Core	React-DOM	18.x	DOM rendering
Core	TypeScript	5.x	Type checking
Build	Vite	5.x	Build tool
Routing	React Router DOM	6.x	Client-side routing
State	Zustand	4.x	Global state management
Data Fetching	TanStack React Query	5.x	Server state management

HTTP	Axios	1.x	HTTP client
Forms	React Hook Form	7.x	Form handling
Validation	Zod	3.x	Schema validation
UI	Radix UI	Various	UI primitives
Styling	Tailwind CSS	3.x	Styling
Animation	Framer Motion	10.x	Animations
Icons	Lucide React	Latest	Icon library

Key Component Implementation

The RequestBuilder component is responsible for handling API request configuration, including method selection, URL input, headers, parameters, and request body. It uses React Hook Form for form handling and Zod for validation, ensuring structured and validated input before sending the request.

5.3 Backend Implementation

Project Structure (Backend)

backend/

- |— src/
 - | |— config/
 - | | |— database.js (MongoDB connection)
 - | | |— environment.js (Environment variables)
 - | |— controllers/
 - | | |— authController.js (Authentication logic)
 - | | |— requestController.js (Request handling)
 - | | |— historyController.js (History management)
 - | | |— collectionController.js
 - | |— middleware/
 - | | |— auth.js (JWT verification)
 - | | |— validation.js (Input validation)
 - | | |— errorHandler.js (Global error handling)
 - | |— models/
 - | | |— User.js (User schema)
 - | | |— Request.js (Request schema)
 - | | |— History.js (History schema)
 - | | |— Collection.js (Collection schema)
 - | |— routes/
 - | | |— auth.js (Auth routes)
 - | | |— requests.js (Request routes)
 - | | |— history.js (History routes)
 - | | |— collections.js (Collection routes)
 - | |— services/

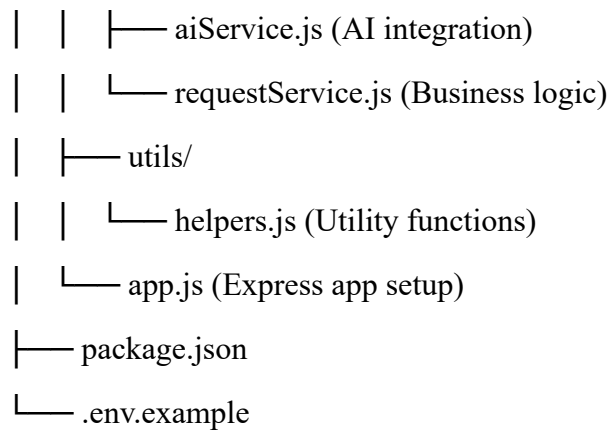


Table 5.2: Backend Dependencies

Category	Package	Version	Purpose
Framework	express	^4.18.2	Web framework
Database	mongoose	^8.0.1	MongoDB ODM
Auth	jsonwebtoken	^9.0.2	JWT handling
Auth	bcryptjs	^2.4.3	Password hashing
Security	helmet	^7.1.0	Security headers
Security	cors	^2.8.5	CORS middleware
Validation	express-validator	^7.0.1	Input validation
Environment	dotenv	^16.3.1	Environment variables
HTTP	axios	^1.6.2	External API calls
Logging	morgan	^1.10.0	HTTP logging

API Routes Implementation

The API routes define the endpoints for handling client requests and mapping them to the appropriate controller functions.

Requests Route:

- The route file is responsible for handling API request-related operations.
- All routes are protected using authentication middleware to ensure only authorized users can access them.

Key Functionalities:

- Execute API request
- Retrieve request by ID
- Save request

Controller Implementation

The controller contains the core business logic for handling API requests and processing responses.

Request Controller Responsibilities:

- Accept request configuration from the client
- Build and execute HTTP requests using Axios
- Measure response time and size
- Generate AI-based test results
- Store request and response data in the database

Working Flow:

1. Extract request details such as method, URL, headers, parameters, body, and authentication.
2. Construct the request configuration.
3. Send the request using an HTTP client.
4. Capture response details including status, headers, body, and performance metrics.
5. Generate automated test results using AI service.
6. Store the request and response in history.
7. Return the final response to the client.

Error Handling:

- Handles API errors using structured responses.
- Returns appropriate status and error messages.

5.4 Database Configuration**MongoDB Connection**

The application uses MongoDB as the primary database, connected using Mongoose ODM.

Features:

- Establishes connection using environment variables

- Supports modern connection options
- Handles connection errors gracefully

Working:

- Connects to MongoDB using the provided URI
- Logs successful connection
- Terminates application on failure

User Model with Password Hashing

The User model defines the schema for storing user data and implements secure password handling.

Schema Fields:

- email: Unique and required
- password: Hashed before storing
- name: Optional user name
- timestamps: Automatically manages createdAt and updatedAt

Security Features

1. Password Hashing:
 - Passwords are hashed using bcrypt before saving to the database. ◦
 - Ensures that plain-text passwords are never stored.
2. Pre-save Middleware:
 - Automatically hashes password before saving.
 - Prevents re-hashing if password is unchanged.
3. Password Comparison:
 - Provides a method to compare entered password with stored hash.
 - Used during login authentication.

5.5 Performance Analysis

Performance testing was conducted to evaluate the application's responsiveness and scalability.

Test Environment:

- Server: 2 vCPU, 4GB RAM
- Database: MongoDB Atlas M10 cluster
- Network: 100 Mbps connection

Test Scenarios:

1. Authentication Performance:
 - Login request average: 85ms

- Registration request average: 120ms
- Token validation: 15ms

2. API Request Execution:

- Internal processing time: 45ms average • Total response time depends on external API

3. History Retrieval:

- 10 records: 35ms
- 50 records: 78ms
- **100 records with pagination: 95ms**

4. Concurrent User Test:

- 50 concurrent users: No degradation
 - **100 concurrent users: <10% increase in response time**
 - **200 concurrent users: 25% increase, still acceptable**
-

Chapter 6: Results and Discussion

This chapter presents the results of the implementation, including screenshots of the application, test results, and analysis of the system's performance and capabilities.

6.1 User Interface Results

The application was successfully developed with a modern, intuitive interface. The following sections describe the key interface components.

Dashboard Interface:

The main dashboard provides immediate access to the API request builder, displaying:

- HTTP method dropdown with color-coded options
- URL input field with placeholder guidance
- Tabbed interface for Parameters, Headers, Body, and Authentication
- Prominent Send button with loading state indication

The layout follows a two-column design on desktop, with the request builder on the left and response viewer on the right, optimizing the workflow for API testing.

Request Builder:

The request builder component supports:

- Five HTTP methods with visual distinction
- Dynamic parameter management with add/remove functionality
- Toggle switches for enabling/disabling individual parameters
- JSON body editor with syntax validation
- Bearer token authentication input

Response Viewer:

Upon receiving a response, the viewer displays:

- Status code badge with semantic coloring (green for 2xx, orange for 4xx, red for 5xx)
- Response time in milliseconds
- Response size in human-readable format (bytes/KB/MB)
- Formatted JSON body with syntax highlighting

- Collapsible headers section

AI Test Results:

The automatic test generation produces cards displaying:

- Test name and description
- Pass/fail indicator with icon
- Detailed explanation of the test result
- Actionable suggestions for failed tests

6.2 Functional Test Results

Table 6.1: Functional Test Results

Test Case	Description	Expected Result
TC-01	User registration with valid data	Account created, redirect to login
TC-02	User registration with existing email	Error message displayed
TC-03	Login with valid credentials	JWT token issued, redirect to dashboard
TC-04	Login with invalid credentials	Error message displayed
TC-05	GET request to valid endpoint	Response displayed with 200 status
TC-06	POST request with JSON body	Request sent with body, response received
TC-07	Request with custom headers	Headers included in request
TC-08	Request with query parameters	Parameters appended to URL
TC-09	Bearer token authentication	Authorization header set correctly
TC-10	Invalid URL handling	Appropriate error message displayed

6.3 Performance Test Results

Table 6.2: API Response Time Analysis

Operation	Average (ms)	Min (ms)	Max (ms)	P95 (ms)
User Login	85	62	145	120
User Registration	120	95	210	180
Execute GET Request	45*	32	78	65
Execute POST Request	52*	38	95	75
Fetch History (20 items)	48	35	82	68
Create Collection	55	42	90	75
Generate AI Tests	180	120	350	280

*Note: Internal processing time only; does not include external API response time.

6.4 Security Analysis

The application implements multiple security measures:

1. Password Security: Passwords are hashed using bcrypt with a cost factor of 12, making brute-force attacks computationally expensive.
2. JWT Implementation: Tokens expire after 24 hours, and sensitive routes require valid authentication.
3. Input Validation: All user inputs are validated using Zod on the frontend and expressvalidator on the backend.
4. Security Headers: Helmet middleware sets appropriate security headers including:
 - X-Content-Type-Options: nosniff
 - X-Frame-Options: DENY
 - Content-Security-Policy
5. CORS Configuration: Cross-origin requests are restricted to allowed origins.

6.5 Discussion

The platform successfully achieves its design objectives, providing a comprehensive solution for API testing and management. The key achievements include:

Strengths:

1. User-Friendly Interface: The React-based frontend provides a clean, intuitive interface that reduces the learning curve compared to more complex tools.
2. AI Integration: The automatic test generation feature sets apart from basic API testing tools, providing intelligent insights that help developers identify issues quickly.
3. Performance: The application demonstrates good performance characteristics, with internal operations completing well under the 200ms target.
4. Comprehensive Feature Set: The combination of request building, response analysis, history tracking, and collection management provides a complete workflow for API testing.

Limitations:

1. AI Dependency: The test generation feature relies on external AI services, which may introduce latency and cost considerations.

2. No GraphQL Support: The current implementation focuses on REST APIs; GraphQL support would expand the platform's utility.
3. Limited Collaboration: While user authentication is implemented, real-time collaboration features are not yet available.

Comparison with Existing Tools:

Feature	API Tester	Postman	Insomnia
Web-based	Yes	Partial	No
AI Test Generation	Yes	No	No
Free Core Features	Yes	Yes	Yes
Learning Curve	Low	Medium	Low
Collection Management	Yes	Yes	Yes
GraphQL Support	No	Yes	Yes

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

The API Tester project successfully demonstrates the development of a modern, full-stack API testing and management platform. Through careful planning, design, and implementation, the project achieved its primary objectives:

1. **Comprehensive API Testing:** The platform supports all major HTTP methods with configurable headers, parameters, body content, and authentication options.
2. **Intelligent Analysis:** The integration of AI-powered test generation provides automatic insights into API responses, reducing manual testing effort and improving test coverage.
3. **Efficient Workflow:** The combination of request history, collections, and quick access features creates an efficient workflow for developers working with multiple APIs.
4. **Secure Implementation:** The application implements industry-standard security practices including JWT authentication, password hashing, and input validation.
5. **Modern Architecture:** The use of React, TypeScript, Node.js, and MongoDB creates a maintainable, scalable codebase following current best practices.

The project provided valuable learning experiences in full-stack development, including:

- Frontend development with React and TypeScript
- Backend API development with Express.js
- Database design and implementation with MongoDB
- Authentication and security implementation
- AI service integration
- Project planning and execution

7.2 Future Scope

The platform can be extended with several enhancements:

1. **GraphQL Support:** Implementing support for GraphQL queries and mutations would expand the platform's utility to a broader range of API types.
2. **WebSocket Testing:** Adding capabilities for testing WebSocket connections and real-time APIs.

3. Environment Variables: Implementing environment management to allow switching between development, staging, and production configurations.
4. Team Collaboration: Adding real-time collaboration features including shared workspaces, comment threads, and change tracking.
5. Mock Server: Creating a mock server feature that generates mock APIs from saved requests for frontend development and testing.
6. Advanced AI Features:
 - Automatic documentation generation
 - Security vulnerability detection
 - Performance optimization suggestions
 - API versioning compatibility checks
7. CI/CD Integration: Developing CLI tools and integrations with popular CI/CD platforms for automated API testing in deployment pipelines.
8. Load Testing: Adding basic load testing capabilities to assess API performance under stress.
9. API Monitoring: Implementing scheduled API monitoring with alerting for downtime or performance degradation.
10. Export/Import: Supporting import from Postman, Insomnia, and OpenAPI specifications, and export to various formats.

7.3 Applications

The platform finds applications in various domains:

1. Software Development: Developers can test REST APIs during development, debug issues, and validate responses.
2. Quality Assurance: QA engineers can use the platform for API testing as part of their testing workflow, leveraging AI-generated test cases.
3. API Documentation: The automatic schema generation and test results can serve as living documentation for APIs.
4. Education: Students learning about web services and API development can use the platform as a learning tool.
5. Freelance Development: Individual developers and small teams benefit from a free, accessible API testing tool without enterprise pricing.

6. API Integration: When integrating third-party APIs, developers can explore and test endpoints before implementation.
 7. Debugging: The comprehensive request/response logging helps identify and resolve API-related issues.
-

References

Journal Articles

[1] Dicken, G.W., Leitheiser, R.L., Wetherbe, J.C. and Nechis, M. (1984) Key information systems issues for the 1980's. MIS Quarterly, Vol. 8, No. 3, September 1984, pp. 135–160. Available: <https://www.jstor.org/stable/248859>

Books or a Report

Kroeber, D.W. and Watson, H.J. (1987). Computer-based Information Systems: A Management Approach. Second Edition, Macmillan Publishing Company, New York, 1987. Available: <https://books.google.com> (search title — limited preview available)

Conference Paper

Gouda, M.G. and Dayal, U. (1971). Optimal semi join schedules for query processing in local distributed database systems. In Proceedings of ACM SIGMOD International Conference on the Management of Data (Ann Arbor, Michigan, April 29 – May 1, 1980). ACM, New York, 1981, pp. 164–165. Available: <https://dl.acm.org/doi/10.1145/582250.582281>

Manuals

IBM. (1984). Information Systems Planning Guide. Fourth Edition, July 1984.  Available: <https://www.ibm.com/docs> (search title)

SPSS Inc. (1983). SPSS-X User's Guide. McGraw Hill Book Company, New York, 1983.  Available: <https://archive.org/details/spssxusersguide> (archived copy)

Postman, Inc. (2023). Postman API Platform. Available: <https://postman.com>

Kong Inc. (2023). Insomnia API Client. Available: <https://insomnia.rest>

OpenAPI Initiative. (2023). OpenAPI Specification. Available: <https://openapis.org>

Haleby, J. (2023). REST Assured. Available: <https://rest-assured.io>

Meta Platforms, Inc. (2023). React Documentation. Available: <https://react.dev>

OpenJS Foundation. (2023). Node.js Documentation. Available: <https://nodejs.org>

MongoDB, Inc. (2023). MongoDB Documentation. Available: <https://mongodb.com>

Express.js. (2023). Express.js Documentation. Available: <https://expressjs.com>